Analyzing Biological Processes Over Time

From Field Observations to Decisions

Claudio D. Silva Jr

09/09/2026

Contents

Chapter 1

Welcome to the workshop!

1.1 What will you learn?

- Recognize three approaches for modeling disease progression over time.
- Experiment with these approaches and learn their basics.
- Obtain and interpret quantities of biological interest.
- Compare what can (and cannot) be learned from each approach.

1.2 Workshop schedule:

10 minutes - Introduction

10 minutes - Comparing the methods

20 minutes - Guided example

20 minutes - Hands-on

20 minutes - Discussion

The complete workshop material, code, and data are available on GitHub.

Chapter 2

Pre-workshop

2.1 How to make the most of this workshop?

- Ask questions! Do not take unresolved doubts home, they stink!
- Use the material afterwards. This material was created to be used as a guide post workshop, use it!
- Code is important, but our focus today is on concepts and interpretation. The code is provided so you can focus on understanding what the models are doing.

2.2 Things to keep in mind:

- This is an introduction, and therefore the focus is you having experience with these models and what they can produce.
- Our example data are considerably well behaved. Real disease progress data can be much messier. We will discuss some of these complications, but solving them is beyond today's scope.
- Coding is not our focus in this workshop, all code is provided. We will focus on three different models and their results.
- Please, keep in mind we have future workshops that expand in different parts of this workshop:
  - Fundamentals of linear mixed models for designed experiments -> 09/19 and 09/20

    * Registration open: Fundamentals of Linear Mixed Models for Designed Experiments
  - Nonlinear models for the plant sciences -> Spring 2027
    * Material from 2026 available online: Nonlinear Models for the Plant Sciences

2.3 Notation guide

We can write the same linear model using different notations, but preserving the same meaning. In this workshop, we will primarily use scalar and probabilistic notation for clarity. Vector and matrix notation are included here for reference. All notations below assume normally distributed errors.

$$\varepsilon_i \sim N(0, \sigma^2)$$

The Multivariate Normal distribution (MVN) is a compact way to represent multiple normally distributed observations together. This distribution shows up when we pile multiple random variables together in vector and matrix notation.

Scalar notation - Represents a single number

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \varepsilon_i \sim N(0, \sigma^2)$$

Where:

- β_0 represents the intercept, the value of y_i when $x_i = 0$;
- β_1 represents the slope, the change in y_i caused by changing one unit in x_i;
- ε_i represents the residual, which we assume to arise from a normal distribution with mean/expectation 0 and variance σ^2;
- y_i represents the response variable and x_i represents the predictor variable.

Vector notation - Represents a vector of numbers

$$\mathbf{y} = \beta_0 \mathbf{1} + \beta_1 \mathbf{x} + \varepsilon \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix} = \beta_0 * \begin{bmatrix} 1 \\ 1 \\ \dots \\ 1 \end{bmatrix} + \beta_1 * \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \dots \\ \varepsilon_n \end{bmatrix} \varepsilon \sim MVN(\mathbf{0}, \sigma^2 \mathbf{I})$$

Here, the variance $\sigma^2\mathbf{I}$ represents the variance covariance matrix, where $\mathbf{I}$ is the identity matrix. For example, for a 3x3 identity matrix, this is how it looks like:

$$\sigma^2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

With this matrix, we assume no correlation between our observed y_i, in other words, independence. This can change by changing the matrix selected. A more general notation is σ^2 , where represents a general matrix.

Matrix notation - Represents a matrix

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon\varepsilon \sim MVN(\mathbf{0}, \sigma^2\mathbf{I})$$

Probabilistic notation - Represents the distribution that we are assuming, a clear way to make our assumptions transparent. The expected value, or mean, can be expressed using any of the previous notations. To follow the same structure of the workshop, scalar notation will be exemplified.

In distributional notation, see how we demonstrate that the deterministic mathematical equation influences the expectation/mean. The variance corresponds to the error term previously introduced as $\varepsilon_i \sim N(0, \sigma^2)$.

This notation reads: Observations y_i arise from a normal distribution with the expectation/mean μ_i controlled by our mathematical equation and a given variance σ^2. This variance relates to the observational level variance.

This notation emphasizes that the model is a statement about how data are generated, combining a deterministic structure with stochastic variability.

$$y_i \sim N(\mu_i, \sigma^2)\mu_i = \beta_0 + \beta_1 x_i$$

Models are Assumptions + Structure + Uncertainty!

2.4 Glossary

Expectation = Mean - Where the distribution balances.

Uncertainty = Degree of doubt or unpredictability - How unsure we are about that estimated value. Can refer to predictions or parameters.

Estimation = The process of obtaining numerical values for the unknown quantities (parameters) of a model from observed data.

Inference = Interpreting estimated parameters in the context of the population or data-generating process (model). Inference is made about non-observable quantities and involves quantifying uncertainty.

Prediction = New data generated by the model, given covariates. A function of the model and stochastic variability.

Data generating process = Process that we believe to generate our data. Involves mathematical model + stochastic component.

Deterministic = Always produces the same result, not random.

Stochastic = Random.

Constraint = A restriction or boundary placed on model parameters. Often based on biological, physical, or logical consideration.

Random variable = Numerical value arising from a probability distribution. Formally: A function that connects the outcome of a random experiment to real numbers.

Support of a probability distribution = Values a random variable arising from this distribution can assume.

Probability distribution = Mathematical description of the probabilities of events/results from a given sample space (all possible outcomes of a random experiment).

Chapter 3

Introduction and applied example

3.1 Let's work together

The experiment:

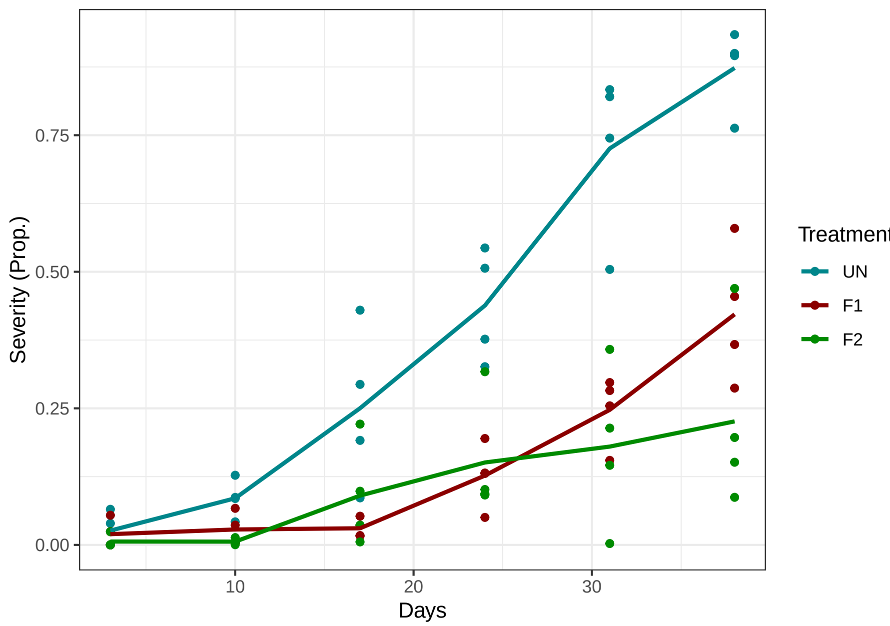
- RCBD (Randomized Complete Block Design) with 4 replicates (blocks)
- Three treatments: Untreated (UN), Fungicide 1 (F1), Fungicide 2 (F2)
- Evaluations over time in 6 different dates after a critical growth stage (Flag Leaf Emergence): 3, 10, 17, 24, 31, and 38 days after
- Disease was measured as: Severity (Percentage of leaf area affected by the disease)*
- The disease: Wheat stripe rust (Polycyclic - Multiple cycles within a crop season)*

***Most critical information**

The data:

```
# Read the data
df <- read_excel("Example_df1.xlsx")

# Control trt factor order
df <- df %>%
  mutate(
```

```
    trt = as.factor(trt),
    trt = fct_relevel(trt, c("UN", "F1", "F2")))

# Exploratory plot
ggplot(df, aes(day, y, colour = trt)) +
  geom_point() +
  stat_summary(geom = "line", fun = mean, size = 1, aes(group = trt)) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Severity (Prop.)", x = "Days", colour = "Treatment") +
  theme_bw()
```



How would you analyse this?

- Pick one time and proceed with ANOVA + Multiple comparison
    - Why not use repeated measures ANOVA or Split-plot in time?
- Average treatments over time and proceed with ANOVA + Multiple comparison
    - Why then collect all this information if we are going to "lose" it?

How about modeling the relationship between time and the response?

3.2 Modeling biological processes in time

Today, we will work with three very accessible options:

3.2.1 Polynomial regression

Polynomial regression is the name given to a linear model whose the standard linear predictor $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ is replaced by a polynomial function:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + ... + \beta_d x_i^d + \varepsilon_i$$

Although polynomial regression is still a linear model[1] it is more flexible than a linear predictor, and allow for producing extremely nonlinear curves (higher the degree d, higher the flexibility).

3.2.2 Beta regression

A beta regression is a linear regression model where the response is assumed to follow a beta distribution[2], while the mean is modeled through a link function.

Recall the linear regression model:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \varepsilon_i \sim Normal(0, \sigma_\varepsilon^2)$$

We can re-write this model using probabilistic notation as:

$$y_i \sim Normal(\beta_0 + \beta_1 x_i,\, \sigma_\varepsilon^2)$$

Now, for a beta regression model, we would write it as:

[1] Linear model: Linear in the parameters. In summary: All unknown parameters only multiplies known quantities; All unknown parameters are added together; All parameters are outside nonlinear functions.

$$\sum_j^b \beta_j xi$$

[2] A beta distribution is a distribution whose the support (values a random variable arising from the distribution can assume) is bounded between 0 and 1. It is commonly used to model variables bounded between this interval, such as proportions.

$$\theta \sim Beta(\alpha,\, \beta),\, \theta \in (0,\, 1)$$

$$y_i \sim Beta(\mu_i,\, \phi)$$

where μ_i is the mean and ϕ is the dispersion parameter, inversely related to the variance. For more on beta regression, check this paper out: Ferrari and Cribari-Neto (2004).

Notice that in both cases, we are modeling the mean (μ), but there is a very important difference, in the beta regression, we model the mean through a link function[3]:

$$\eta_i = g(\mu_i) = \beta_0 + \beta_1 x_i$$

Here, our link function will be the logit:

$$g(\mu_i) = log(\frac{\mu_i}{1-\mu_i})$$

This link function transforms proportions from (0, 1) to the real line. And our expectation/mean is:

$$\mu_i = logit^{-1}(\eta_i) = \frac{1}{1+e^{-\eta_i}}$$

Which is our original response scale at (0, 1).

Why would we use beta regression if it is similar to a linear regression?

Remark: The standard beta distribution does not include exact 0 or 1, and such values can happen. Handling these boundary values is an extension beyond this workshop. Options include:

- Hurdle models:
- Transformations: Smithson and Verkuilen (2006)

3.2.3 Logistic Disease Progress Curve

Disease progress curves are nonlinear models historically used in plant pathology to model the progress of plant disease epidemics on time. They are mechanistic

[3] A link function is a function that maps the expected value (mean) of the response variable to the linear predictor.

$$g(\mu) = \eta$$

models[4] with direct relation to specific types of disease cycles, with logistic models being associated with polycyclic diseases, and monomolecular models to monocyclic diseases.

Among the disease progress curves, the logistic is one of the most widely used ones!

The logistic disease progress curve is represented with the following equation:

$$y_i = \frac{K}{1 + (\frac{K - y_0}{y0})e^{-rx_i}}$$

However, we use the logistic disease progress curve equation to model the expectation/mean of our distribution, similarly to what we did before. Today, we will be doing something like this:

$$y_i \sim N(\mu_i, \sigma^2)\mu_i = \frac{K}{1 + (\frac{K - y_0}{y0})e^{-rx_i}}$$

In this model, it is important to notice that parameters have meaning, the following points describe the three parameters of the logistic disease progress curve:

- K - Maximum disease potential, upper asymptote.
- y_0 - Initial disease level at $t = 0$.
- r - Apparent rate of growth: Epidemic "speedometer", how fast the epidemic moves from y_0 to K.

Remark: They are not mutually exclusive!

What are your thoughts on why we chose these options?

3.3 Applying the logistic disease progress curve to our example

This is the model we will be working with:

$$y_{ij} \sim N(\mu_{ij}, \sigma^2)\mu_{ij} = \frac{K_i}{1 + (\frac{K_i - y_{0i}}{y_{0i}})e^{-r_i t_j}}$$

[4]Mechanistic models are designed to account for underlying processes in the system being studied. These underlying processes are represented through the model parameters. Each parameter has naturally a biological meaning, or a direct interpretation.

where the mean μ_{ij} of the ith treatment levels at the jth day was modeled using the logistics disease progress curve, with the three parameters K, r, and y_0, being specific for each ith treatment level.

The research question we wish to answer is: Which product was the most effective in suppressing the epidemic?

```
# Getting initials for the model
treatments <- unique(df$trt)

initials <- data.frame()

for(i in seq_along(treatments)){
  df_f <- filter(df, df$trt == treatments[i])

  init <- getInitial(y ~ SSlogis(day, Asym, xmid, scal), df_f)

  temp <- data.frame(
    K = as.numeric(init["Asym"]),
    r = 1/as.numeric(init["scal"]),
    treat = treatments[i])

  temp$y0 <- temp$K/(exp(as.numeric(init["xmid"])*temp$r)+1)

  initials <- rbind(initials, temp)
}

# Model fit
## Cehck treatment levels - UN as reference
levels(df$trt)

## Organize starting values
init_UN <- filter(initials, treat == "UN")
init_F1 <- filter(initials, treat == "F1")
init_F2 <- filter(initials, treat == "F2")

start <- c(
  # K
  init_UN$K,
  init_F1$K - init_UN$K,
  init_F2$K - init_UN$K,

  # r
  init_UN$r,
  init_F1$r - init_UN$r,
  init_F2$r - init_UN$r,
```

```
  # y0
  init_UN$y0,
  init_F1$y0 - init_UN$y0,
  init_F2$y0 - init_UN$y0
)

## Fitting the model using generalized least squares
m1 <- gnls(
  y ~ K / (1 + ((K - y0) / y0) * exp(-r * day)),
  data = df,
  params = K + r + y0 ~ trt,
  start = start
)
```

What did we get:

Estimates:

$$K \; r \; y_0$$

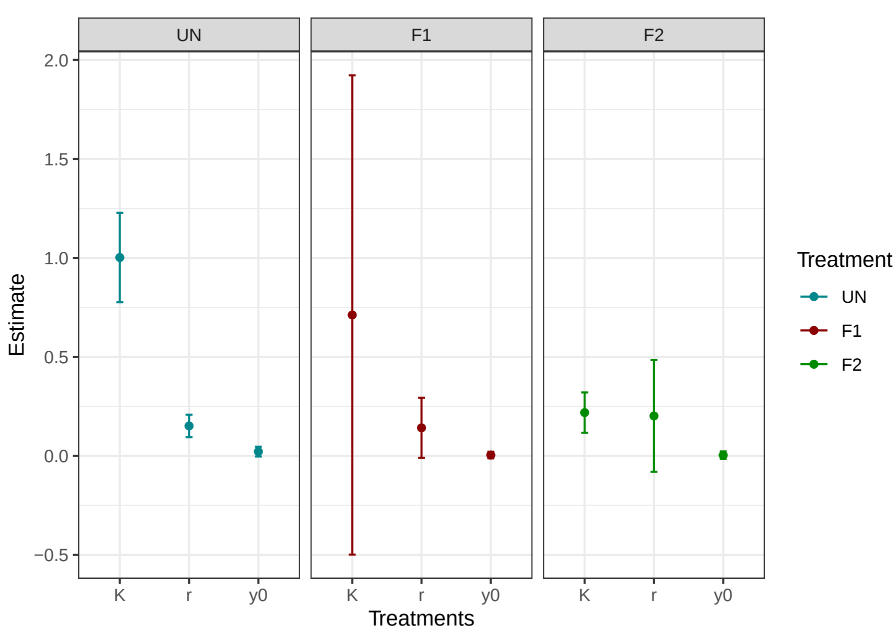
```
m1_est <- data.frame(
  trt = c("UN", "F1", "F2"),
  K_est = c(coef(m1)[1], coef(m1)[1] + coef(m1)[2], coef(m1)[1] + coef(m1)[3]),
  r_est = c(coef(m1)[4], coef(m1)[4] + coef(m1)[5], coef(m1)[4] + coef(m1)[6]),
  y0_est = c(coef(m1)[7], coef(m1)[7] + coef(m1)[8], coef(m1)[7] + coef(m1)[9]))

# SE for parameter estimates
m1_est <- m1_est %>%
  mutate(
    K_se = case_when(
      trt == "UN" ~ sqrt(vcov(m1)[1,1]),
      trt == "F1" ~ deltamethod(~ x1+x2, coef(m1), vcov(m1)),
      trt == "F2" ~ deltamethod(~ x1+x3, coef(m1), vcov(m1))),

    r_se = case_when(
      trt == "UN" ~ sqrt(vcov(m1)[4,4]),
      trt == "F1" ~ deltamethod(~ x4+x5, coef(m1), vcov(m1)),
      trt == "F2" ~ deltamethod(~ x4+x6, coef(m1), vcov(m1))),

    y0_se = case_when(
      trt == "UN" ~ sqrt(vcov(m1)[7,7]),
      trt == "F1" ~ deltamethod(~ x7+x8, coef(m1), vcov(m1)),
      trt == "F2" ~ deltamethod(~ x7+x9, coef(m1), vcov(m1)))
    )
```

```r
# CI for K
m1_est$K_lb <- m1_est$K_est - 1.96*m1_est$K_se
m1_est$K_ub <- m1_est$K_est + 1.96*m1_est$K_se

# CI for r
m1_est$r_lb <- m1_est$r_est - 1.96*m1_est$r_se
m1_est$r_ub <- m1_est$r_est + 1.96*m1_est$r_se

# CI for r
m1_est$y0_lb <- m1_est$y0_est - 1.96*m1_est$y0_se
m1_est$y0_ub <- m1_est$y0_est + 1.96*m1_est$y0_se

est_long <- m1_est %>%
  pivot_longer(cols = -trt, names_to = c("par", ".value"), names_sep = "_")


# Plot
est_long <- est_long %>%
  mutate(
    trt = as.factor(trt),
    trt = fct_relevel(trt, "UN", "F1", "F2"))

summary(m1)
```

```
## Generalized nonlinear least squares fit
##   Model: y ~ K/(1 + ((K - y0)/y0) * exp(-r * day))
##   Data: df
##         AIC       BIC   logLik
##   -134.4891 -111.7224 77.24454
##
## Coefficients:
##                     Value Std.Error   t-value p-value
## K.(Intercept)   1.0018741 0.1153627  8.684559  0.0000
## K.trtF1        -0.2900662 0.6281129 -0.461806  0.6458
## K.trtF2        -0.7830126 0.1265064 -6.189508  0.0000
## r.(Intercept)   0.1513961 0.0291060  5.201542  0.0000
## r.trtF1        -0.0094225 0.0828558 -0.113722  0.9098
## r.trtF2         0.0504394 0.1468550  0.343464  0.7324
## y0.(Intercept)  0.0219960 0.0126578  1.737743  0.0871
## y0.trtF1       -0.0173268 0.0152994 -1.132518  0.2617
## y0.trtF2       -0.0182585 0.0160080 -1.140586  0.2584
##
##  Correlation:
##                K.(In) K.trF1 K.trF2 r.(In) r.trF1 r.trF2 y0.(I) y0.tF1
```

```
## K.trtF1         -0.184
## K.trtF2         -0.912  0.167
## r.(Intercept)   -0.836  0.154  0.763
## r.trtF1          0.294 -0.909 -0.268 -0.351
## r.trtF2          0.166 -0.030 -0.425 -0.198  0.070
## y0.(Intercept)   0.715 -0.131 -0.652 -0.967  0.340  0.192
## y0.trtF1        -0.592  0.583  0.540  0.800 -0.799 -0.159 -0.827
## y0.trtF2        -0.566  0.104  0.659  0.764 -0.269 -0.732 -0.791  0.654
##
## Standardized residuals:
##         Min          Q1         Med          Q3         Max
## -2.34282142 -0.48021522 -0.07631176  0.37522827  2.89725703
##
## Residual standard error: 0.08847691
## Degrees of freedom: 72 total; 63 residual
```

```
ggplot(est_long, aes(par, est, colour = trt)) +
  geom_point() +
  geom_errorbar(aes(ymin = lb, ymax = ub), width = 0.1) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Estimate", x = "Treatments", colour = "Treatment") +
  facet_wrap(~trt) +
  theme_bw()
```



```r
kable(est_long)
```

trt	par	est	se	lb	ub
UN	K	1.0018741	0.1153627	0.7757632	1.2279850
UN	r	0.1513961	0.0291060	0.0943484	0.2084439
UN	y0	0.0219960	0.0126578	-0.0028133	0.0468053
F1	K	0.7118079	0.6174280	-0.4983509	1.9219668
F1	r	0.1419736	0.0775753	-0.0100740	0.2940212
F1	y0	0.0046692	0.0085936	-0.0121743	0.0215127
F2	K	0.2188615	0.0519166	0.1171050	0.3206179
F2	r	0.2018356	0.1439417	-0.0802902	0.4839614
F2	y0	0.0037375	0.0097998	-0.0154701	0.0229451

```r
summary(m1)

cont <- rbind(
  "K: F1 - F2" = c(0, 1, -1, 0, 0, 0, 0, 0, 0),
  "r: F1 - F2" = c(0, 0, 0, 0, 1, -1, 0, 0, 0),
  "y0: F1 - F2" = c(0, 0, 0, 0, 0, 0, 0, 1, -1))

summary(glht(m1, linfct = cont))
```

Contrasts:

$$\Delta = \phi_i - \phi_j H_0 : \Delta = 0 H_a : \Delta \neq 0$$

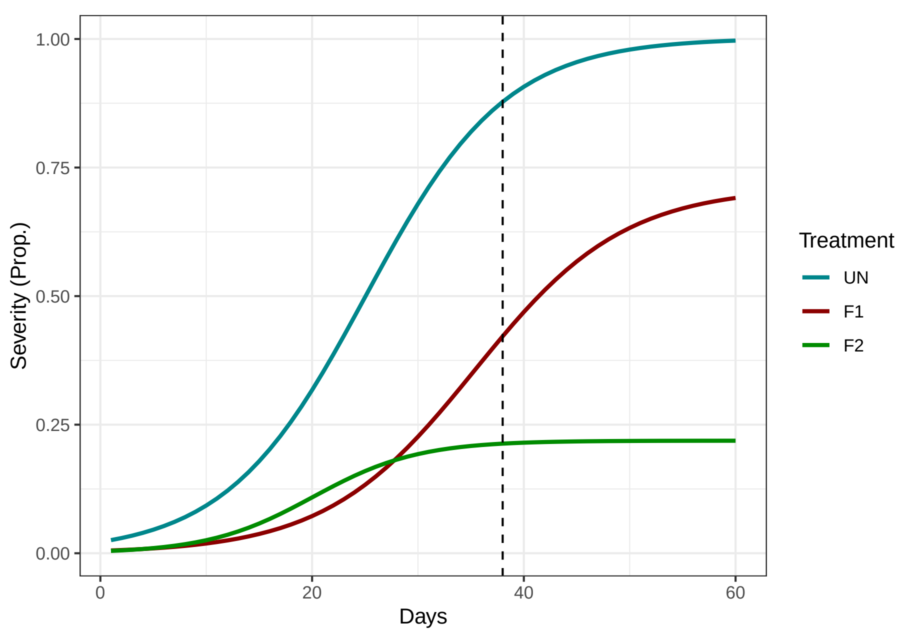
```r
contrasts_df <- data.frame(
  contrast = c("UN - F1", "UN - F2", "F1 - F2"),
  K = c(0.646, "< 0.001", 0.675),
  r = c(0.910, 0.732, 0.933),
  y0 = c(0.262, 0.258, 0.999)
)

kable(contrasts_df)
```

contrast	K	r	y0
UN - F1	0.646	0.910	0.262
UN - F2	$<$ 0.001	0.732	0.258
F1 - F2	0.675	0.933	0.999

Predicted curves:

$$\mu_{ij}$$

```
pred_df <- data.frame(
  trt = c(rep("UN", 60), rep("F1", 60), rep("F2", 60)),
  day = c(rep(seq(1, 60, by = 1), 3)))

pred_df$pred <- predict(m1, pred_df)

pred_df <- pred_df %>%
  mutate(
    trt = as.factor(trt),
    trt = fct_relevel(trt, "UN", "F1", "F2"))

ggplot(pred_df, aes(day, pred, colour = trt, group = trt)) +
  geom_line(linewidth = 1) +
  geom_vline(xintercept = 38, linetype = 2) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Severity (Prop.)", x = "Days", colour = "Treatment") +
  theme_bw()
```



How many units of y are increasing per day for each curve at the maximum growth point[5]?

$$\frac{Kr}{4}$$

[5]The LDPC has a different slope at every time, but it reaches a maximum in $\frac{K}{2}$.

```r
m1_est$y_inf <- m1_est$K_est/2

m1_est$max_s <- m1_est$r_est*m1_est$y_inf*(1-(m1_est$y_inf/m1_est$K_est))

UN_se_slope <- deltamethod(~ (x1*x4)/4, coef(m1), vcov(m1)) # UN se

F1_se_slope <- deltamethod(~ ((x1+x2)*(x4+x5))/4, coef(m1), vcov(m1)) # F1 se

F2_se_slope <- deltamethod(~((x1+x3)*(x4+x6))/4, coef(m1), vcov(m1)) # F2 se

m1_est <- m1_est %>%
  mutate(se = case_when(
    trt == "UN" ~ UN_se_slope,
    trt == "F1" ~ F1_se_slope,
    trt == "F2" ~ F2_se_slope))

m1_est$max_s_lb <- m1_est$max_s-1.96*m1_est$se
m1_est$max_s_ub <- m1_est$max_s+1.96*m1_est$se

slope_table <- m1_est[,-c(2:13)]

kable(slope_table)
```

trt	y_inf	max_s	se	max_s_lb	max_s_ub
UN	0.5009371	0.0379200	0.0043548	0.0293845	0.0464555
F1	0.3559040	0.0252645	0.0104325	0.0048167	0.0457123
F2	0.1094307	0.0110435	0.0063841	-0.0014692	0.0235563

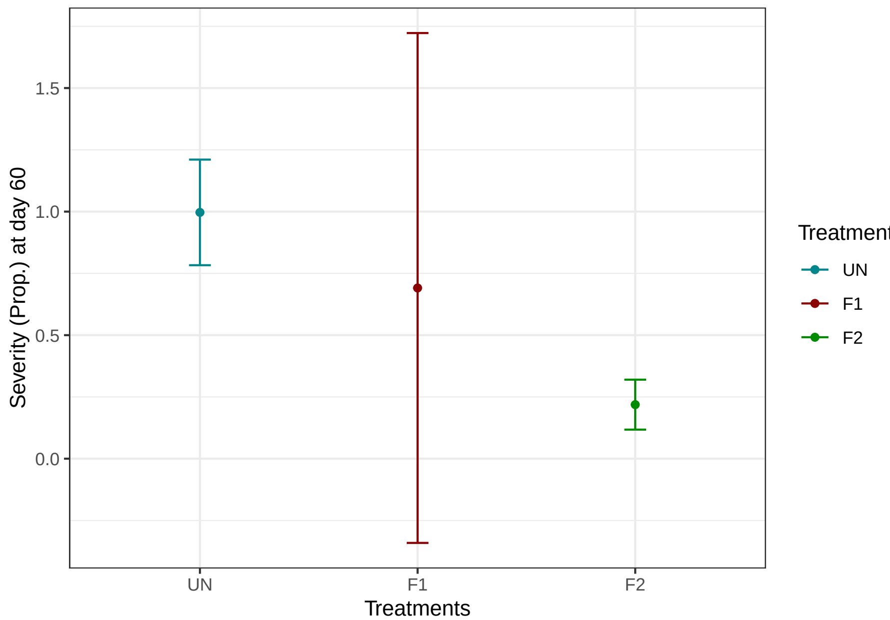
```r
slop_cont <- data.frame(
  contrast = c("UN-F1", "UN-F2", "F1-F2"),
  z = c(

    (slope_table$max_s[1] - slope_table$max_s[2])/(deltamethod(~ (x1*x4)/4 - ((x1+x2)*

    (slope_table$max_s[1] - slope_table$max_s[3])/(deltamethod(~ (x1*x4)/4 - ((x1+x3)*

    (slope_table$max_s[2] - slope_table$max_s[3])/(deltamethod(~ ((x1+x2)*(x4+x5))/4 -

    )
)

slop_cont$p_value <- 2*pnorm(abs(slop_cont$z), lower.tail = FALSE)

kable(slop_cont)
```

contrast	z	p__value
UN-F1	1.119462	0.2629433
UN-F2	3.477836	0.0005055
F1-F2	1.162713	0.2449462

Remark: The logistic disease progress curve has multiple slopes, we are looking at the maximum here!

What is the disease severity at day 60?

```
pred_60 <- filter(pred_df, day == 60)

UN_se_60 <- deltamethod(~ (x1 / (1 + ((x1 - x7) / x7) * exp(-x4 * 60))), coef(m1), vcov(m1))

F1_se_60 <- deltamethod(~ ((x1+x2) / (1 + (((x1+x2) - (x7+x8)) / (x7+x8)) * exp(-(x4+x5) * 60))),

F2_se_60 <- deltamethod(~ ((x1+x3) / (1 + (((x1+x3) - (x7+x9)) / (x7+x9)) * exp(-(x4+x6) * 60))),

pred_60 <- pred_60 %>%
  mutate(se = case_when(
    trt == "UN" ~ UN_se_60,
    trt == "F1" ~ F1_se_60,
    trt == "F2" ~ F2_se_60))

pred_60$CI.ub <- pred_60$pred+1.96*pred_60$se
pred_60$CI.lb <- pred_60$pred-1.96*pred_60$se

ggplot(pred_60, aes(trt, pred, colour = trt)) +
  geom_point() +
  geom_errorbar(aes(ymin = CI.lb, ymax = CI.ub), width = 0.1) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Severity (Prop.) at day 60", x = "Treatments", colour = "Treatment") +
  theme_bw()
```



Conclusion:

- Fungicide 2 was capable of reducing the maximum severity potential, limiting the potential disease damage;
- Fungicide 2 also reduced the maximum disease increase compared to untreated check;
- There is no evidence of differences between Fungicide 1 and the untreated check;
- 60 days after the epidemic started, expected/mean proportion is relatively different between treatments. However, we are much more certain of a high severity for UN and low for F2 than the severity for the F1 treatment.

Interesting topics of discussion:

- Notice how uncertainty is propagated from the estimated curve parameters and the calculated derived quantities[6].
- Most of the lack of evidence about the differences between UN and F1, and UN and F2, arise from the high uncertainty on F1's maximum disease potential. It does not mean our analysis is wrong, but it exposes one of the difficulties of estimating multiple parameters of nonlinear models.
  - More about this on Nonlinear Models for the Plant Sciences

[6] Derived quantities are functions of estimated parameters.

Chapter 4

Hands-on: Polynomial regression

This is the model we will be working with:

$$y_{ijk} = \beta_{0i} + \beta_{1i}x_j + \beta_{2i}x_j^2 + \beta_{3i}x_j^3 + b_k + \varepsilon_{ijk} b_k \sim N(0, \sigma_b^2) \varepsilon_{ijk} \sim N(0, \sigma^2)$$

This is a cubic polynomial. The response mean is modeled as a function of the ith level of the fungicide treatment at the jth day, with a random deviation caused by the kth block.

We could also write this in distributional notation, as a marginal distribution:

$$y_{ijk} \sim N(\beta_{0i} + \beta_{1i}x_j + \beta_{2i}x_j^2 + \beta_{3i}x_j^3,\ \sigma_b^2 + \sigma^2)$$

Here we are treating block as a random effect, which means that our observed blocks are a sample from a larger group (population) of blocks, and that the block effect causes a random deviation in our response mean.

You can learn more about polynomials in here: James et al. (2021)

Script and data here!

Questions:

- Get predicted curves and check then. (pred_df2)
- Check the derived quantities, what do you think of the inflection point in y (y_inf) and the maximum slope (max_s)? (der_df2_vis)

- Check the contrasts of the maximum slope, couple that with the predicted curves, which treatment is being more effective in suppressing the epidemic? (contrasts: cont_s2; curves: pred_df2)
- What the prediction at day 60 tell you? Does it make sense? (pred60_2)

```r
# Packages
library(readxl) # Reading excel files
library(tidyverse) # Wrangling
library(msm) # Delta method
library(lme4) # Linear mixed models
library(emmeans) # emmeans for mult comp

# Read the data
df <- read_excel("Example_df1.xlsx")

# Control trt factor order
df <- df %>%
  mutate(
    trt = as.factor(trt),
    trt = fct_relevel(trt, c("UN", "F1", "F2")))

# Fit the model
m2 <- lmer(y ~ trt*poly(day, 3, raw = TRUE) + (1|block), data = df)

# Check model parameters - We will not look into model parameters or their contrasts h
summary(m2)

# Obtain predicted curves + Plot
pred_df2 <- data.frame(
  trt = as.factor(c(rep("UN", 60), rep("F1", 60), rep("F2", 60))),
  day = c(rep(seq(1, 60, by = 1), 3)))

pred_df2$trt <- fct_relevel(pred_df2$trt, c("UN", "F1", "F2"))

pred_df2$pred <- predict(m2, newdata = pred_df2, re.form = NA)

ggplot(pred_df2, aes(day, pred, colour = trt, group = trt)) +
  geom_line() +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  theme_bw() +
  labs(y = "Severity (Prop.)", x = "Days", colour = "Treatment")

# Obtain the maximum slope

## Obtaining the quantities of interest (derived quantities)
```

```r
### Parameter values of each treatment level
der_df2 <- data.frame(
  trt = c("UN", "F1", "F2"),
  b0 = c(fixef(m2)[1], fixef(m2)[1] + fixef(m2)[2], fixef(m2)[1] + fixef(m2)[3]),
  b1 = c(fixef(m2)[4], fixef(m2)[4] + fixef(m2)[7], fixef(m2)[4] + fixef(m2)[8]),
  b2 = c(fixef(m2)[5], fixef(m2)[5] + fixef(m2)[9], fixef(m2)[5] + fixef(m2)[10]),
  b3 = c(fixef(m2)[6], fixef(m2)[6] + fixef(m2)[11], fixef(m2)[6] + fixef(m2)[12])
)

### x_inflection
der_df2$x_inf <- -(der_df2$b2/(3*der_df2$b3))

### Which disease severity corresponds to the maximum (y_inflection)?
der_df2 <- der_df2 %>%
  mutate(y_inf = case_when(
    trt == "UN" ~ pred_df2[pred_df2$day == 25 & pred_df2$trt == "UN", 3],
    trt == "F1" ~ pred_df2[pred_df2$day == 60 & pred_df2$trt == "F1", 3],
    trt == "F2" ~ pred_df2[pred_df2$day == 21 & pred_df2$trt == "F2", 3]))

### max_slope - For F1, since b3 > 0, we will use the maximum within the range 0 - 60 for x, whic
der_df2 <- der_df2 %>%
  mutate(max_s = case_when(

    trt %in% c("UN", "F2") ~ (der_df2$b1 + der_df2$b2*(2*der_df2$x_inf) + der_df2$b3*(3*(der_df2$

    trt == "F1" ~ (der_df2$b1 + der_df2$b2*(2*60) + der_df2$b3*(3*(60)^2))))

der_df2 <- der_df2 %>%
  mutate(se = case_when(
    trt == "UN" ~ deltamethod(~ x4 + x5*(2*24.7) + x6*(3*(24.7)^2), fixef(m2), vcov(m2)),
    trt == "F1" ~ deltamethod(~ (x4 + x7) + (x5 + x9)*(2*60) + (x6 + x11)*(3*(60)^2), fixef(m2),
    trt == "F2" ~ deltamethod(~ (x4 + x8) + (x5 + x10)*(2*20.6) + (x6 + x12)*(3*(20.6)^2), fixef(

der_df2_vis <- der_df2[, c(1, 6, 8, 7, 9)]

der_df2_vis$max_s_ci.ub <- der_df2_vis$max_s + 1.96*der_df2_vis$se
der_df2_vis$max_s_ci.lb <- der_df2_vis$max_s - 1.96*der_df2_vis$se

print(der_df2_vis) # Visualization

#### Max slope contrast
cont_s2 <- data.frame(
  contrast = c("UN-F1", "UN-F2", "F1-F2"),
  z = c(
```

```
    (der_df2$max_s[1] - der_df2$max_s[2])/(deltamethod(~ (x4 + x5*(2*24.7) + x6*(3*(24

    (der_df2$max_s[1] - der_df2$max_s[3])/(deltamethod(~ (x4 + x5*(2*24.7) + x6*(3*(24

    (der_df2$max_s[2] - der_df2$max_s[3])/(deltamethod(~ ((x4 + x7) + (x5 + x9)*(2*60)

  )
)

cont_s2$p_value <- round(2*pnorm(abs(cont_s2$z), lower.tail = FALSE), 4)

# Predicted disease severity at day 60
pred60_2 <- data.frame(emmeans(m2, ~ trt | day, at = list(day = 60)))

ggplot(pred60_2, aes(trt, emmean, colour = trt)) +
  geom_point() +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = 0.1) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Severity (Prop.) at day 60", x = "Treatments", colour = "Treatment") +
  theme_bw()
```

Chapter 5

Hands-on: Beta regression

This is the model we will be working with:

$$y_{ijk} \mid b_k \sim Beta(\mu_{ijk}, \phi)\mu_{ijk} = \eta_{ijk}^{-1} = \frac{1}{1 + e^{-(\beta_{0i} + \beta_{1i}t_j + b_k)}} b_k \sim N(0, \sigma_b^2)$$

We can also denote the link function as:

$$\eta_{ijk} = g(\mu_{ijk}) = log(\frac{\mu_{ijk}}{1 - \mu_{ijk}}) = logit(\mu_{ijk})$$

Where the response mean is modeled as a function of the ith level of the fungicide treatment at the jth day, with a random deviation caused by the kth block.

Here we are treating block as a random effect, which means that our observed blocks are a sample from a larger group (population) of blocks, and that the block effect causes a random deviation in our response mean. In the model, $y \mid b$ indicates conditional distribution, in other words, the distribution of y after we fix/observe a level of block b.

More about it here: Fundamentals of Linear Mixed Models for Designed Experiments

You can also learn more about Generalized Linear Models (GLMs) here: Stroup et al. (2024)

Script and data here!

Questions:

- Check the slopes on the link scale, what they mean? Do you think the contrast results makes sense? (slopes: slopes3; contrast: slope_link_cont)

- Get predicted curves and check then. Check both at the link scale (pred_df3$pred_l) and response scale (pred_df3$pred_p).
- Check the derived quantities, what do you think of the inflection point in y (y_inf) and the maximum slope (max_s)? (der_df3)
- Check the contrasts of the maximum slope, couple that with the predicted curves in the response scale, which treatment is being more effective in suppressing the epidemic? (contrasts: cont_s3; curves: pred_df3$pred_p)
- Did the contrast change from the slopes at the link scale and the slopes at the response scale?
- What the prediction at day 60 tell you? Does it make sense? (pred60_3)

```
# Packages
library(readxl) # Reading excel files
library(tidyverse) # Wrangling
library(msm) # Delta method
library(lme4) # Linear mixed models
library(emmeans) # emmeans for mult comp
library(glmmTMB) # beta regression / glmm

# Read the data
df <- read_excel("Example_df1.xlsx")

# Control trt factor order
df <- df %>%
  mutate(
    trt = as.factor(trt),
    trt = fct_relevel(trt, c("UN", "F1", "F2")))

# Fit the model
m3 <- glmmTMB(y ~ trt*day + (1|block), data = df, family = beta_family(link = "logit")

# Check model parameters
summary(m3)

# Obtain predictions + plot
pred_df3 <- data.frame(
  trt = as.factor(c(rep("UN", 60), rep("F1", 60), rep("F2", 60))),
  day = c(rep(seq(1, 60, by = 1), 3)))

pred_df3$trt <- fct_relevel(pred_df3$trt, c("UN", "F1", "F2"))

pred_df3$pred_l <- predict(m3, newdata = pred_df3, re.form = NA, type = "link") # link
pred_df3$pred_p <- predict(m3, newdata = pred_df3, re.form = NA, type = "response") #
```

```r
ggplot(pred_df3, aes(day, pred_p, colour = trt, group = trt)) +
  geom_line() +
#  geom_point(aes(day, pred_l, colour = trt)) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  theme_bw() +
  labs(y = "Severity (Prop.)", x = "Days", colour = "Treatment")

# Slope

## We can compare the slope on the link scale:
slopes_link <- emtrends(m3, ~ trt, var = "day")
slope_link_cont <- pairs(slopes_link, adjust = "none")

slopes3 <- data.frame(slopes_link)

## We can also find the max slopes at the response scale, and compare then. As a shortcut, here t
der_df3 <- data.frame(
  trt = c("UN", "F1", "F2"),
  y_inf = c(0.5, 0.5, 0.5),
  x_inf = c(-fixef(m3)$cond[1]/slopes3$day.trend[1], -(fixef(m3)$cond[1]+fixef(m3)$cond[2])/slope
  max_s = c(slopes3$day.trend[1]/4, slopes3$day.trend[2]/4, slopes3$day.trend[3]/4),
  se = c(slopes3$SE[1]/4, slopes3$SE[2]/4, slopes3$SE[3]/4),
  CI.lb = c(slopes3$asymp.LCL[1]/4, slopes3$asymp.LCL[2]/4, slopes3$asymp.LCL[3]/4),
  CI.ub = c(slopes3$asymp.UCL[1]/4, slopes3$asymp.UCL[2]/4, slopes3$asymp.UCL[3]/4)
)

### Max slope contrast
cont_s3 <- data.frame(
  contrast = c("UN-F1", "UN-F2", "F1-F2"),
  z = c(

    (der_df3$max_s[1] - der_df3$max_s[2])/(deltamethod(~ x4/4 - (x4+x5)/4, fixef(m3)$cond, vcov(m

    (der_df3$max_s[1] - der_df3$max_s[3])/(deltamethod(~ x4/4 - (x4+x6)/4, fixef(m3)$cond, vcov(m

    (der_df3$max_s[2] - der_df3$max_s[3])/(deltamethod(~ (x4+x5)/4 - (x4+x6)/4, fixef(m3)$cond, v

  )
)

cont_s3$p.value <- 2*pnorm(abs(cont_s3$z), lower.tail = FALSE)

# Predicted disease severity at day 60
pred60_3 <- data.frame(emmeans(m3, ~ trt | day, at = list(day = 60), type = "response"))
```

```r
ggplot(pred60_3, aes(trt, response, colour = trt)) +
  geom_point() +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.1) +
  scale_color_manual(values = c("turquoise4", "#8B0000", "green4")) +
  labs(y = "Severity (Prop.) at day 60", x = "Treatments", colour = "Treatment") +
  theme_bw()
```

Chapter 6

Discussion

6.1 How the results change?

- LDPC was the only method that allowed us to check the epidemic meaningful parameters K, y_0, and r.
- Beta and polynomial regression do not have parameters with direct biological meaning.
- We calculated derived quantities with similar meanings across methods, let's check then together.

Inflection points:

Der.Q	Trt	LDPC	Poly	Beta
y_inflection	UN	0.5009371	0.5022986	0.5
y_inflection	F1	0.3559040	1.3948893	0.5
y_inflection	F2	0.1094307	0.1139722	0.5

- The inflection of a logistic is bounded between y_0 and K. When y_0 represents the disease level at $t = 0$ (approximately 0), ensuring $y0 < K/2$, the inflection happens at $K/2$.
- The inflection of the polynomial regression has no bounds and is determined by the estimated coefficients, it is not constrained by the time of observation or the response range.
- The inflection point is determined by the choice of link function and predictor. With a logit link and a linear predictor, the inflection occurs at $\mu = 0.5$. These two components determine the trajectory shape. Additionally, the conditional mean is bounded within (0, 1) given the support of the Beta distribution.

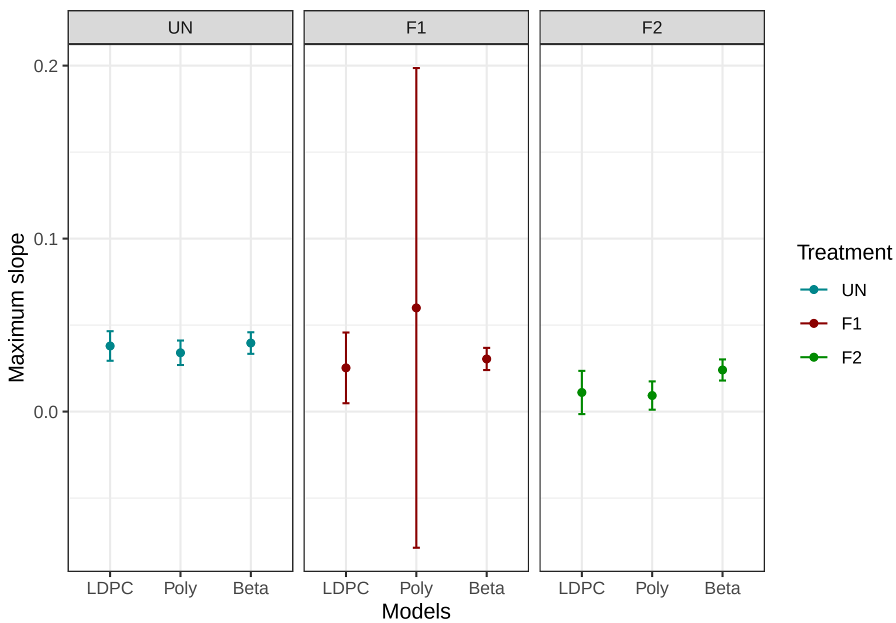
  - Notice something important: The inverse logit (our μ) is the same function as the logistic regression:

$$\frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}}$$

- Why not compare inflection in x? We could, but recall the polynomial regresion:
  - We assumed that the inflection for F1 was at x = 60, bounding it between 0 and 60, but we can't know for sure once the function is convex (happy face).

trt	x_inf	max_s	y_inf	se	max_s_ci.ub	max_s_ci.lb
UN	24.69112	0.0339961	0.5022986	0.0036146	0.0410807	0.0269114
F1	-42.66029	0.0599295	1.3948893	0.0707342	0.1985686	-0.0787096
F2	20.63571	0.0092756	0.1139722	0.0041679	0.0174446	0.0011066

Maximum slopes:



- For treatments such as "UN" and "F2", where the data show clearer patterns, the three modeling approaches produce very similar estimates of maximum slope.

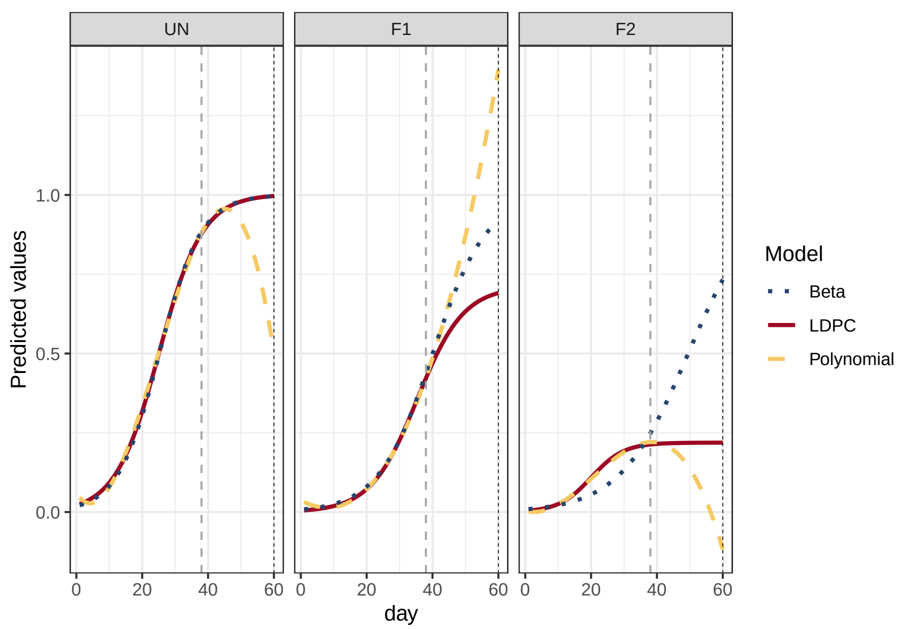
- Comparing beta regression and the LDPC, the second shows greater uncertainty. This may reflect differences in model structure, the LDPC estimates three parameters that jointly determine the trajectory and maximum slope, whereas the beta regression with a linear predictor has a simpler mean structure.

$$Slope_{max,\ LDPC} = \frac{Kr}{4} Slope_{max,\ Beta} = \frac{\beta_1}{4}$$

- Derived quantities can be more sensitive to model choice than the curves themselves!
- For F1, where the data provide considerably less information about the trajectory, uncertainty in the polynomial maximum slope becomes very large. The flexibility and lack of biological constraints of the polynomial allow this uncertainty to propagate strongly into the derived quantity.

We are looking at the result of information in the data + model structure choice + assumption choice

Predicted curves and prediction for day 60:



- LDPC is constrained by K, the curve stop growing and approaches the maximum potential at that point (slope = 0);

- The beta regression is constrained between 0 and 1 by the assumed distribution, coupled with the inverse logit, it produces this sigmoidal shape that increases toward 1.
- The polynomial regression is unconstrained, and higher orders can be very flexible. Within the observed range it behaves similar to the LDPC, but outside the observed range it produces nonsense trajectories.
- Important note:
  - LDPC: If $r = 0$, the curve stays at y_0, if $r < 0$, the curve decreases.
  - Beta regression: In the response scale, If $\beta_1 = 0$, the curve stays at β_0, if $\beta_1 < 0$, the curve decreases toward 0.

6.2 Summary

Point of discussion	Logistic DPC	Polynomial regression	Beta regression
Distribution adopted	Normal (Assumed)	Normal (Assumed)	Beta
Mean trajectory	Logistic DPC equation	Cubic polynomial	Link scale: Linear Response scale: Inverse logit
Mean boundaries	y_0 and K	No boundaries	0 - 1 bounded
Upper assymptote	K	None (the sky is the limit!)	1
Direct inference from model parameters	Yes	Generally no	Link scale: Yes Response scale: Maybe
Works well for extrapolation/forecast	Yes	No	Maybe Approaches 0 or 1
Constant slope	No	No	Link scale: Yes Response scale: No

Point of discussion	Logistic DPC	Polynomial regression	Beta regression
Main strenght	Epidemiologically interpretable trajectory	Flexible description	Appropriate support + bounded mean
Feasability	Could be complex to fit, especially when data is sparse, best option for extrapolation.	Easy to fit compare to LDPC, will yield good results within observed range.	Easier and most straight forward option, especially if focusing on inference at link scale.

Chapter 7

References

Ferrari, Silvia, and Francisco Cribari-Neto. 2004. "Beta Regression for Modelling Rates and Proportions." *Journal of Applied Statistics* 31 (7): 799–815. https://doi.org/10.1080/0266476042000214501.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning*. Springer US. https://doi.org/10.1007/978-1-0716-1418-1.

Smithson, Michael, and Jay Verkuilen. 2006. "A Better Lemon Squeezer? Maximum-Likelihood Regression with Beta-Distributed Dependent Variables." *Psychological Methods* 11 (1): 54–71. https://doi.org/10.1037/1082-989x.11.1.54.

Stroup, Walter W., Marina Ptukhina, and Julie Garai. 2024. *Generalized Linear Mixed Models*. March 27. https://doi.org/10.1201/9780429092060.